



JavaScript

Pr. RABHI Loubna

Plan

I- Introduction

II- Les Bases de Javascript

III- Les Objets JavaScript

IV- JavaScript avancé

I- Introduction

Langage Compilé Vs langage de script

- Programmation en langages de bas niveau (proches de la machine)
 - très difficile (complexité)
 - Loin du langage humain
- Besoin de
 - Langages de haut niveau
 - avec des structures et des modèles conceptuels proches de l'humain
 - Programmes de génération des langages bas niveau à partir de langages haut niveau
 - **Compilateurs**
 - **Interpréteurs**
 - ...

I- Introduction

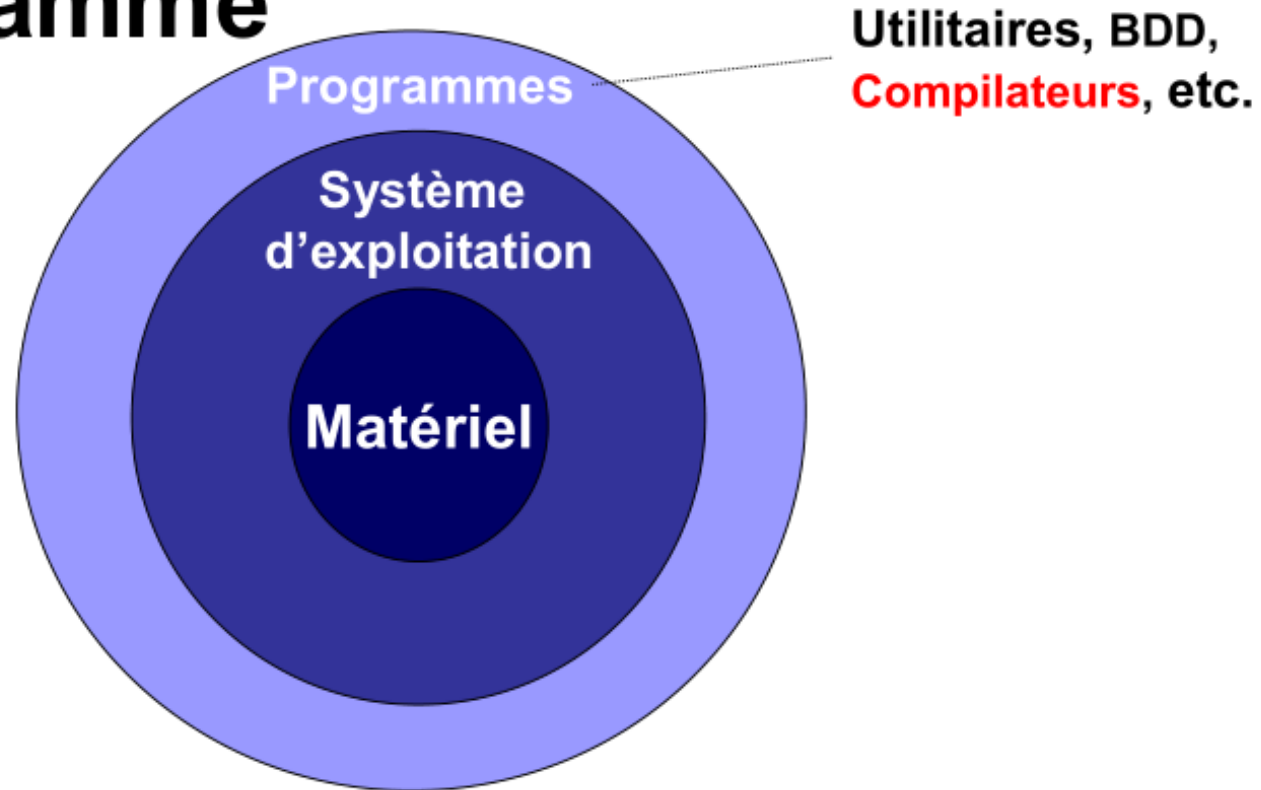
Langage Compilé Vs langage de script

Langage compilé	langage de script / langage interprété
•Un langage compilé est un langage de programmation pour lequel le code source est traduit en langage machine ou en code intermédiaire avant son exécution. •Le processus de compilation consiste à prendre l'intégralité du code source et à le traduire en un fichier exécutable ou en un autre format intermédiaire (comme le bytecode dans le cas de langages comme Java). •Les langages compilés incluent souvent des langages comme C, C++, C#, et Fortran.	Un langage de script est un langage de programmation interprété, ce qui signifie que le code source est exécuté directement par un interpréteur sans passer par une étape de compilation préalable. Les scripts sont généralement écrits dans des langages plus haut niveau et sont souvent utilisés pour automatiser des tâches ou pour des scénarios de développement rapide. Les langages de script incluent Python,, JavaScript, et PHP.

I- Introduction

Langage Compilé Vs langage de script

- Un compilateur est un programme

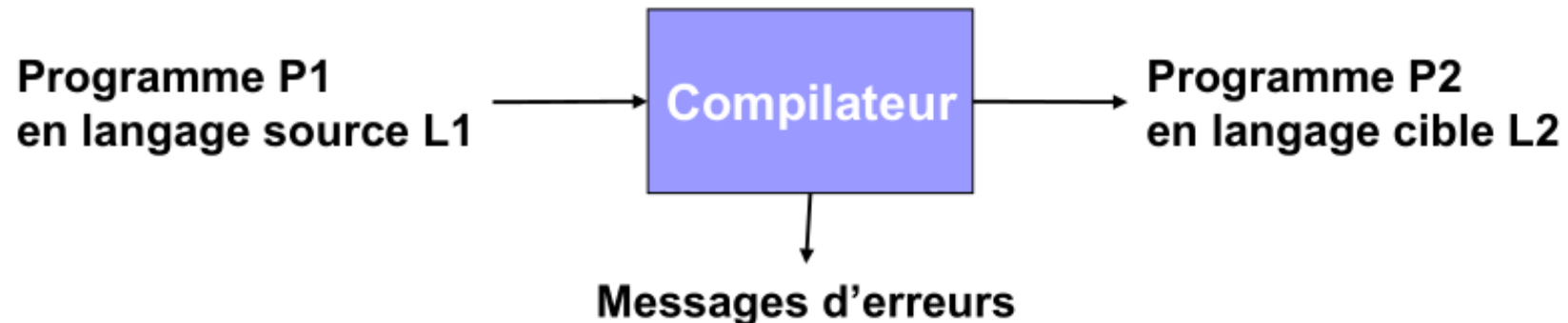


I- Introduction

Langage Compilé Vs langage de script

Compilateur - définition

- Un compilateur est un programme (une fonction) qui prend en entrée un programme dans un langage source L1 et produit en sortie un programme équivalent dans un langage L2

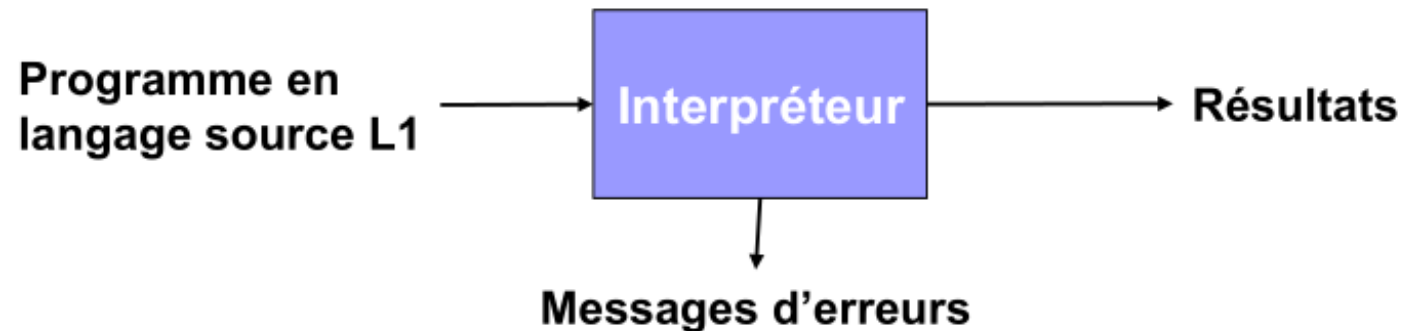


I- Introduction

Langage Compilé Vs langage de script

Interpréteur - définition

- Un interpréteur est programme qui prend en entrée un programme dans un langage source L1 et produit en sortie les résultats de l'exécution de ce programme



I- Introduction

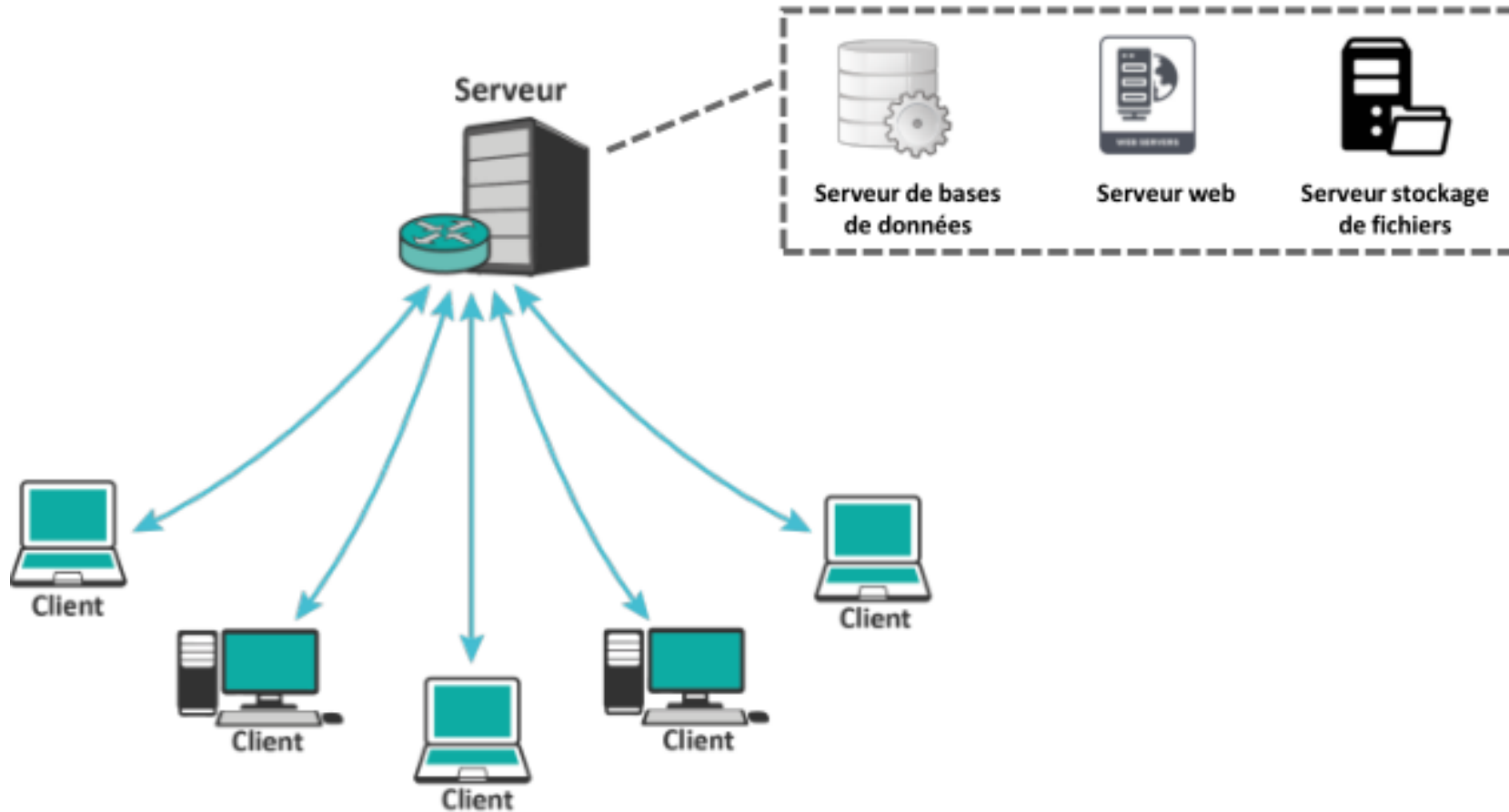
Langage Compilé Vs langage de script

Types d'un langage de script : coté client et coté serveur

- Les langages de script côté serveur s'exécutent sur un serveur Web. Lorsqu'un client envoie une requête, le serveur répond en envoyant du contenu via le protocole HTTP. Les scripts côté serveur ne sont pas visibles par le public. Leur rôle est d'assurer la rapidité du traitement, l'accès aux données et la résolution des erreurs.
- Exemples de langages de script côté serveur :
 - PHP, ASP .NET, Node.js, Java, Ruby, Perl.
- Les langages de script côté client s'exécutent du côté du client, sur son navigateur Web. L'avantage des scripts côté client est qu'ils peuvent réduire la demande sur le serveur, ce qui permet aux pages Web de se charger plus rapidement. Ces scripts sont axés sur l'interface utilisateur et ses fonctionnalités.
- Exemples de langages de script côté client :
 - HTML, CSS, JavaScript.

I- Introduction

Comprendre l'architecture client/serveur



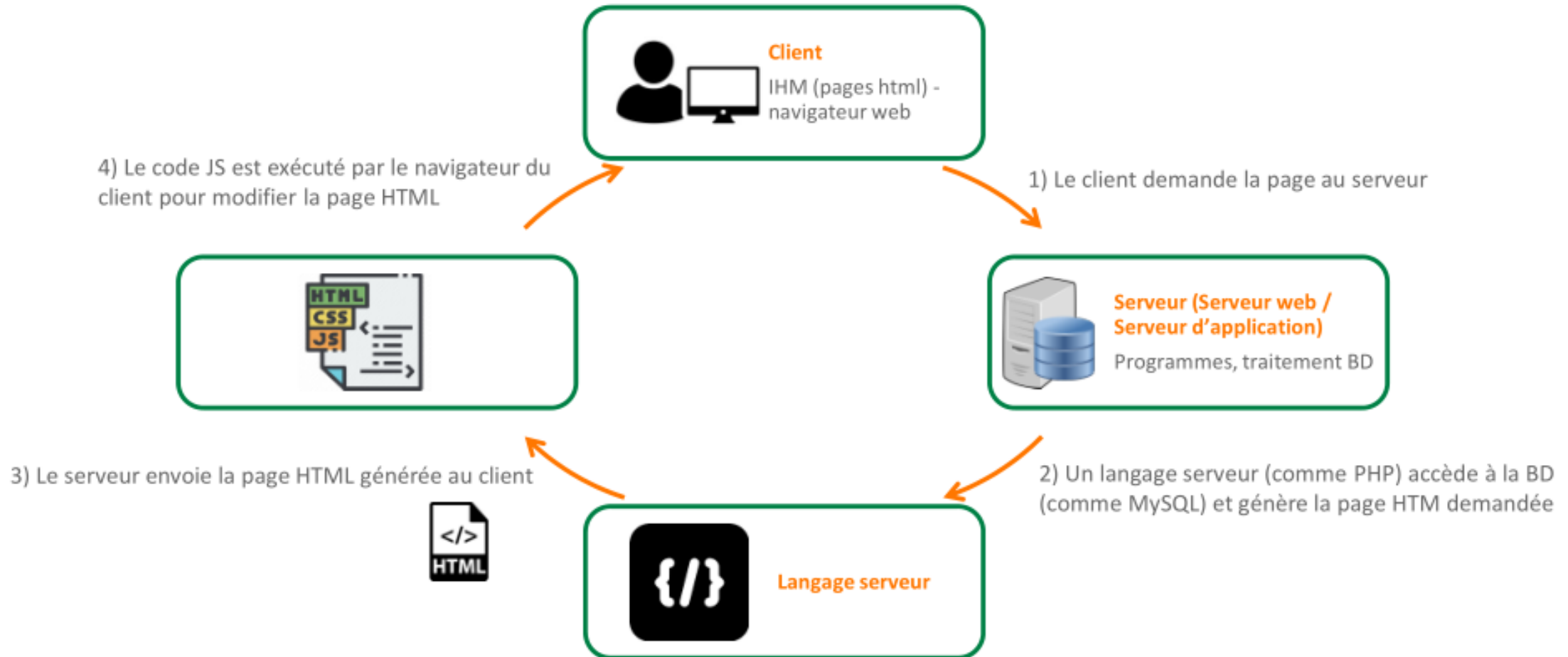
I- Introduction

Comprendre l'architecture client/serveur

- 1 L'utilisateur saisit l'URL (Uniform Resource Locator) du site web ou du fichier. Le navigateur demande alors au serveur le DNS (DOMAIN NAME SYSTEM).
- 2 Le serveur DNS recherche l'adresse du serveur WEB.
- 3 Le serveur DNS répond avec l'adresse IP du serveur Web.
- 4 Le navigateur envoie une requête HTTP/HTTPS à l'adresse IP du serveur WEB (fournie par le serveur DNS).
- 5 Le serveur envoie les fichiers nécessaires du site web (pages html, documents, images, ...).
- 6 Le navigateur rend alors les fichiers et le site web s'affiche. Ce rendu est effectué à l'aide de l'interpréteur DOM (Document Object Model), de l'interpréteur CSS et du moteur JS, collectivement connus sous le nom de compilateurs JIT ou (Just in Time).

I- Introduction

Comprendre l'architecture client/serveur



I- Introduction

Découvrir l'écosystème de développement

Les caractéristiques
que vous devez chercher



- La coloration du code et la mise en évidence de la syntaxe
- Pliage de code
- Bon appariement automatique des Crochets des parenthèses et des guillemets
- Complétion automatique du code
- Permet de naviguer un projet entier
- Bons outils de recherche et de remplacement de caractéristiques

BBEDIT



TextWrangler



Sublime Text



VS CODE



I- Introduction

Découvrir l'écosystème de développement

Front-end vs back-end

Front-End

- Le terme "**front-end**" désigne l'interface utilisateur.
- Le front-end est construit en utilisant une combinaison de technologies telles que le langage de balisage hypertexte (HTML), JavaScript et les feuilles de style en cascade (CSS).

Les frameworks front-end

1. Angular JS
2. React.js
3. JQuery
4. Vue.js

Back-End

- Le terme "**back-end**" désigne le serveur, l'application et la base de données qui travaillent en coulisses pour fournir des informations à l'utilisateur.
- La programmation back-end est définie comme la logique informatique d'un site Web ou d'un logiciel, depuis le stockage et l'organisation des données jusqu'à la création des algorithmes et des séquences logiques complexes qui fonctionnent, d'une manière transparente, sur le front-end.
- Les langages back-end les plus populaires pour sont Ruby, Python, Java, ASP .Net et PHP.

Les frameworks back-end

1. Laravel
2. Node.JS

II- Les Bases de Javascript

Qu'est ce que Javascript?

Historique et environnement
de JavaScript



- JavaScript a été créé en 1995 par Netscape
- S'exécutent directement dans le navigateur
- Créer pour remplacer d'autre solution impuissant
- Etait destiné spécifiquement à être utilisé sur le web
- Crée de l'interactivité
- JavaScript est utilisé plus communément dans le Front-office
- Peut aussi être utilisé en Back-office.

II- Les Bases de Javascript

Qu'est ce que Javascript?



Java	JavaScript
Java est un langage de programmation compilé	JavaScript est un langage de programmation interprété
Les applications Java peuvent s'exécuter sur n'importe quelle machine virtuelle (JVM) ou navigateur	Le code JavaScript est exécuté sur le navigateur uniquement
Les objets de Java sont basés sur les classes, même si nous ne pouvons créer aucun programme en Java sans créer une classe	Les objets JavaScript sont basés sur des prototypes
Java est un langage autonome	JavaScript est inclut dans une page Web et s'intègre à son contenu HTML
Un programme Java utilise plus de mémoire	JavaScript nécessite moins de mémoire, il est donc utilisé dans des pages Web

II- Les Bases de Javascript

Intégration du Javascript dans HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Bienvenu dans Javascript</title>
</head>
<body>

  <script>
    alert("Bienvenu dans Javascript");
  </script>
</body>
</html>
```

1

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Bienvenu dans Javascript</title>
</head>
<body>

  <a href="JavaScript:window.alert('Bienvenu dans JavaScript!');">
    cliquez ici</a>
</body>
</html>
```

3

```
<> index.html × JS script.js ×
C: > Users > mon pc > Desktop > JS > S1 > JS script.js
1 alert("Bienvenu dans Javascript");
```

2

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Bienvenu dans Javascript</title>
</head>
<body>

  <script src="script.js"></script>
</body>
</html>
```


II- Les Bases de Javascript

Les Variables

Variable :

Objet informatique identifié par un « identificateur », d'un type donné, possédant une case mémoire et une valeur qui peut changer au cours de l'exécution du programme.

II- Les Bases de Javascript

Les Variables

Constante :

Objet informatique identifié par un « identificateur », possédant une case mémoire et une valeur qui ne peut pas changer au cours de l'exécution du programme.

II- Les Bases de Javascript

Les Variables

Identificateur :

Chaîne de caractères alphanumérique qui commence par un caractère alphabétique, qui ne contient pas d'espaces et pas de caractères spéciaux (é, è, à, ê, î, ...)



II- Les Bases de Javascript

Les Variables

Exemple d'identificateurs :

-A, a, age, etudiant, Un_Etudiant

Sont des identificateurs corrects

-à, âge, étudiant, 1Etudiant

Ne peuvent pas être des identificateurs

II- Les Bases de Javascript

Les Variables

Keywords and reserved words

break case catch class const continue debugger
default delete do else enum export extends false
finally for function if implements import in
instanceof interface let new null package private
protected public return static super switch this
throw true try typeof var void while with yield

II- Les Bases de Javascript

Les Variables

```
var variable_nom;
```

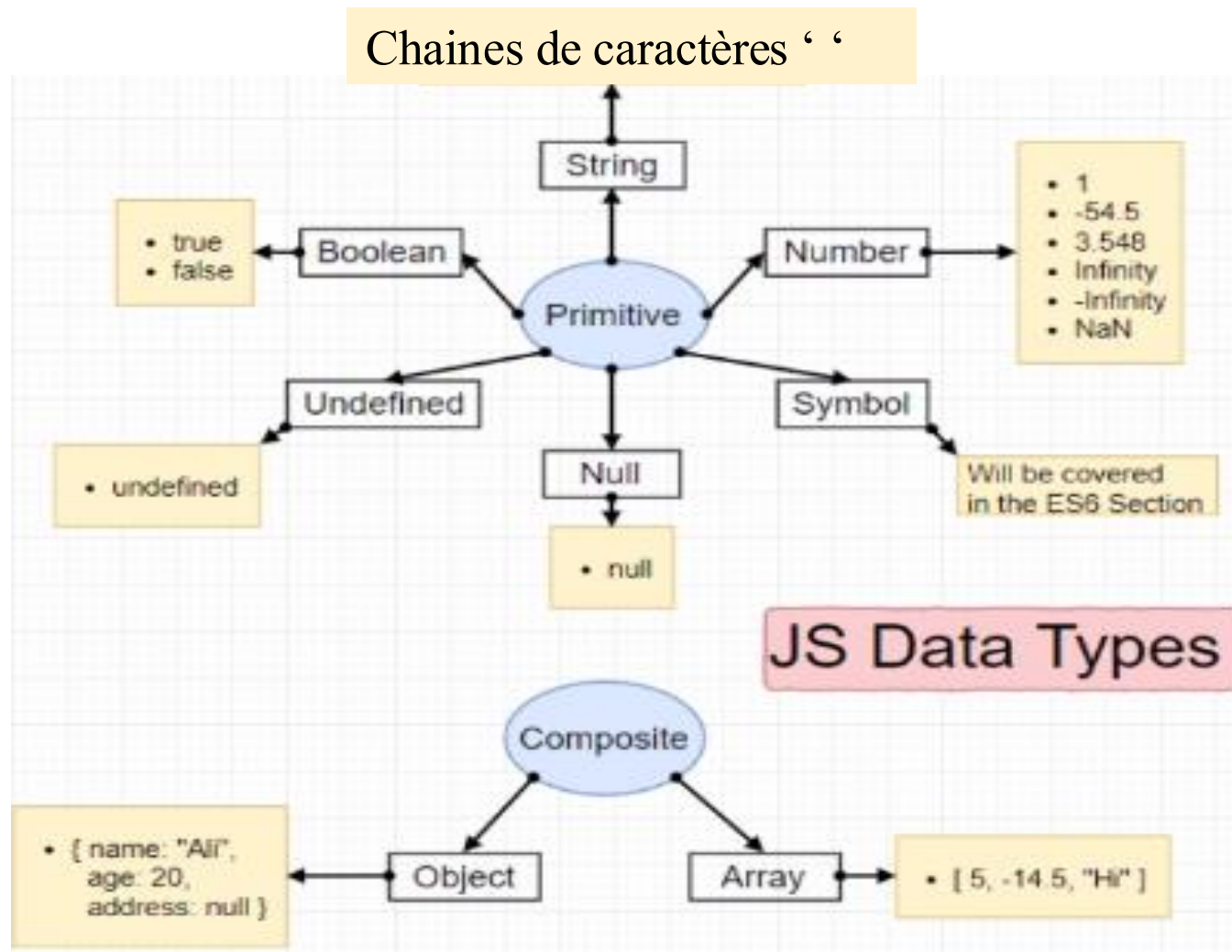
```
let variable_nom;
```

NB:

Les variables **maVariable** et **MaVariable** sont considérées comme deux variables distinctes en raison de la différence de casse.

II- Les Bases de Javascript

Les Variables



II- Les Bases de Javascript

Les Variables

Number type

Certains langages de programmation divisent les variables numériques en integer, float, big float, Mais en JS, tous ces types sont regroupés en type « number »

NaN = Not a Number (pas un nombre)

```
> Infinity+100  
< Infinity  
> Infinity-100  
< Infinity  
> Infinity/100  
< Infinity  
> Infinity*100  
< Infinity  
> Infinity*(-100)  
< -Infinity
```

```
> Infinity-Infinity  
< NaN  
> 0/0  
< NaN  
> Math.sqrt(-1)  
< NaN
```


II- Les Bases de Javascript

Les Variables

String type

```
> "Nous allons apprendre à travailler avec les strings"
< 'Nous allons apprendre à travailler avec les strings'
> 'Nous allons apprendre à travailler avec les strings'
< 'Nous allons apprendre à travailler avec les strings'
> 'Nous allons apprendre à travailler avec les strings'
✖ Uncaught SyntaxError: Invalid or unexpected token
```

```
> 'un string, c'est quoi?'
✖ Uncaught SyntaxError: Unexpected identifier 'est'
> "un string, c'est quoi?"
< "un string, c'est quoi?"
> 'un "string", c'est quoi?'
✖ Uncaught SyntaxError: Unexpected identifier 'est'
> "un \"string\", c'est quoi?"
< `un "string", c'est quoi?`
>
```

On ne peut pas mélanger
l'apostrophe avec le
guillemet

```
> "12"
< '12'
> '12.33333'
< '12.33333'
>
```

II- Les Bases de Javascript

Les Variables

Boolean type

```
> true
< true
> false
< false
> True
✖ ▶ Uncaught ReferenceError: True is not defined
  at <anonymous>:1:1
> TRUE
✖ ▶ Uncaught ReferenceError: TRUE is not defined
  at <anonymous>:1:1
>
```

```
> var maVoiture = "Peugeot";
< undefined
> var monAutreVoiture = "Hyundai";
< undefined
> maVoiture === monAutreVoiture ;
< false
> monAutreVoiture = "Peugeot";
< "Peugeot"
> maVoiture === monAutreVoiture ;
< true
```

II- Les Bases de Javascript

Les Variables

Boolean type

```
> Boolean("Bonjour");
```

```
< true
```

```
> Boolean("");
```

```
< false
```

```
> Boolean(" ");
```

```
< true
```

```
> Boolean(10);
```

```
< true
```

```
> Boolean(0);
```

```
< false
```

```
> Boolean(false);
```

```
< false
```

```
> Boolean("false");
```

```
< true
```

II- Les Bases de Javascript

Les Variables

Boolean type



- Les valeurs Falsy
 - false
 - null
 - undefined
 - ""
 - 0
 - NaN
- Toutes les autres valeurs sont truthy
- Attention aux strings "0" et "false", ils sont considérés comme « true »

II- Les Bases de Javascript

Les Variables

Undefined type

`undefined` = pas de valeur

```
> let x=5;
```

```
< undefined
```

```
> x
```

```
< 5
```

```
> let y;
```

```
< undefined
```

```
> y
```

```
< undefined
```

II- Les Bases de Javascript

Les Variables

Null type

« null » est généralement utilisée pour indiquer l'absence d'une valeur.

null = pas de valeur

null est utilisé pour indiquer l'absence volontaire de valeur. Il est souvent utilisé dans les scénarios où une valeur est inconnue, non définie

```
> var a;  
↵ undefined  
  
> a  
↵ undefined  
  
> var a=null;  
↵ undefined  
  
> a  
↵ null
```

II- Les Bases de Javascript

Les Variables composées

Array type

```
> var a=[true, "hello", 15, false, null, -2.5]
```

```
↳ undefined
```

```
> a[0]
```

```
↳ true
```

```
> a[1]
```

```
↳ 'hello'
```

```
> a[2]
```

```
↳ 15
```

II- Les Bases de Javascript

Les Variables composées

Object type

```
> var o ={name: "Ali", age: 14, Adresse: null};  
⏏ undefined  
⏏  
> o["name"]  
⏏ 'Ali'  
⏏  
> o.name  
⏏ 'Ali'  
⏏  
> o["age"];  
⏏ 14  
⏏  
> o.age  
⏏ 14  
⏏  
> o["Adresse"];  
⏏ null  
⏏  
> o.Adresse  
⏏ null
```


II- Les Bases de Javascript

Les constantes

Constante :

Objet informatique identifié par un « identificateur », possédant une case mémoire et une valeur qui ne peut pas changer au cours de l'exécution du programme.

II- Les Bases de Javascript

Les constantes

```
> const PI=3.14;
```

```
↳ undefined
```

```
> PI
```

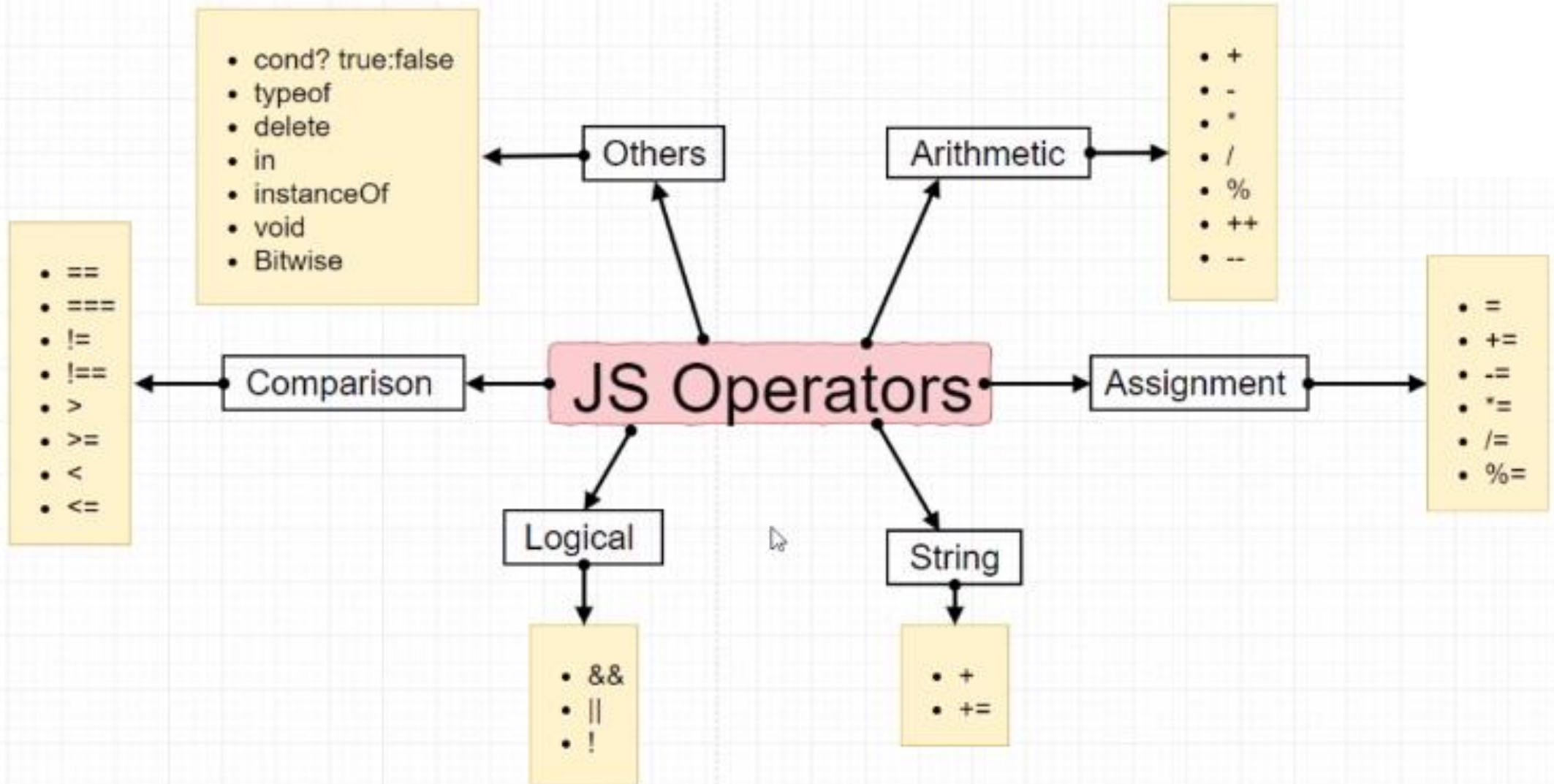
```
↳ 3.14
```

```
> PI=5;
```

```
✖ ▶ Uncaught TypeError: Assignment to constant variable.  
   at <anonymous>:1:3
```

II- Les Bases de Javascript

Expressions et Opérateurs



II- Les Bases de Javascript

Expressions et Opérateurs

Arithmetic

```
> 12+15  
< 27  
> 12-5  
< 7  
> 12*3  
< 36  
> 12/3  
< 4  
> 12%5  
< 2
```

```
> var x=5;  
< undefined  
> x++  
< 5  
> x  
< 6  
> ++x  
< 7  
> x  
< 7
```

```
> x--  
< 7  
> x  
< 6  
> --x  
< 5  
> x  
< 5
```

II- Les Bases de Javascript

Expressions et Opérateurs

Arithmetic

```
> Math.round(12.5)
< 13
> Math.round(12.9)
< 13
> Math.round(12.4)
< 12
> Math.random()
< 0.32433821333056323
> Math.random()
< 0.2999856397151264
> Math.pow(2, 3)
< 8
```

```
> Math.max(2, 12, 7)
< 12
> Math.min(2, 12, 7)
< 2
> Math.sqrt(4)
< 2
> Math.ceil(12.76)
< 13
> Math.ceil(12)
< 12
```

II- Les Bases de Javascript

Expressions et Opérateurs

Assignment

```
> var x=3;  
↵ undefined  
> x  
↵ 3  
> x+=2 //x=x+2  
↵ 5  
> x-=1 //x=x-1  
↵ 4  
> x*=3 // x=x*3  
↵ 12  
> x%=5  
↵ 2  
> x/=2 //x=x/2  
↵ 1
```

II- Les Bases de Javascript

Expressions et Opérateurs

String- Concaténation

```
> var monPrenom = "loubna"
< undefined
> var monNom= "RABHI"
< undefined
> monPrenom+' '+monNom
< 'loubna RABHI'
> monPrenom+=monNom;
< 'loubnaRABHI'
```

II- Les Bases de Javascript

Expressions et Opérateurs

Logical

```
> true && true;
```

```
< true
```

```
> true && false;
```

```
< false
```

```
> false && true;
```

```
< false
```

```
> false && false;
```

```
< false
```

```
> true || true;
```

```
< true
```

```
> true || false;
```

```
< true
```

```
> false || true;
```

```
< true
```

```
> false || false;
```

```
< false
```

```
> !true
```

```
< false
```

```
> !false
```

```
< true
```


II- Les Bases de Javascript

Expressions et Opérateurs

Comparison

```
> 4=="4"  
< true  
  
> 4==="4"  
< false  
  
> 3 != "3"  
< false  
  
> 3 !== "3"  
< true
```

```
> 4<8  
< true  
  
> 4>=4  
< true  
  
> 4>1  
< true  
  
> 4>5  
< false
```

II- Les Bases de Javascript

Expressions et Opérateurs

others

```
> 5>2? 1:2
< 1
> 5<2? 1:2
< 2
> typeof(5)
< 'number'
> typeof("Hello")
< 'string'
> typeof(3>5)
< 'boolean'
> typeof(y)
< 'undefined'
.
```

II- Les Bases de Javascript

Expressions et Opérateurs

Conversion de Type en JS: String to Int (parseInt())

```
> var certifMicrosoft= prompt("Combien de certificats Microsoft avez-vous ");
< undefined
> var certifOracle= prompt("Combien de certificats Oracle avez-vous ");
< undefined
> var certif=certifMicrosoft + certifOracle;
< undefined
> certif;
< '102'
```

```
var certifMicrisoft= prompt("Combien de certificats Microsoft avez-vous ");
var certifOracle= prompt("Combien de certificats Oracle avez-vous ");
var certif= parseInt(certifMicrisoft) + parseInt(certifOracle);
console.log(certif);|
```

II- Les Bases de Javascript

Expressions et Opérateurs

Conversion de Type en JS: String to Int (parseInt())

```
> parseFloat("3.14");  
< 3.14  
> parseFloat("3");  
< 3  
> parseFloat("3.14 Bonjour tout le monde");  
< 3.14  
> parseFloat(" Bonjour 3.14 Bonjour tout le monde");  
< NaN  
> parseInt("3.14");  
< 3  
> parseInt(" bonjour 3.14 bonjour");  
< NaN
```

II- Les Bases de Javascript

Expressions et Opérateurs

Conversion de Type AUTOMATIQUE en JS

```
> "5"+1  
< '51'  
  
> "5"-1  
< 4  
  
> false == 0  
< true  
  
> 3*null  
< 0
```

II- Les Bases de Javascript

Expressions et Opérateurs

Commentaires et espaces blancs

```
1
2 // Ceci est un commentaire qui est ignoré par JavaScript
3
4 var prenom = "François"; // Ceci stocke un prénom
5
6 /* Ceci est un commentaire qui couvre plusieurs lignes.
7    Il utilise la meme syntaxe que le CSS.
8 */
```

II- Les Bases de Javascript

Exercices

Exercice 1 :

Quel est le résultat de l'exécution de chacune de ces lignes dans la console ? Pourquoi ?

1. `var a; typeof a;`
2. `var s = '1s'; s++;`
3. `!!"false"`
4. `!!undefined`
5. `typeof -Infinity`
6. `10 % "0"`
7. `undefined == null`
8. `false === ""`
9. `typeof "2E+2"`
10. `a = 3e+3; a++;`

II- Les Bases de Javascript

Solution

```
> var a; typeof a;
```

```
< 'undefined'
```

```
> var s = '1s'; s++;
```

```
< NaN
```

```
> !! "false"
```

```
< true
```

```
> !!undefined
```

```
< false
```

```
> typeof -Infinity
```

```
< 'number'
```

```
> 10 % "0"
```

```
< NaN
```

```
> undefined == null
```

```
< true
```

```
> false === ""
```

```
< false
```

```
> typeof "2E+2"
```

```
< 'string'
```

```
> a = 3e+3; a++;
```

```
< 3000
```


II- Les Bases de Javascript

Exercices

Exercice 2 :

Donner le résultat logique de chacune des déclarations suivantes :

1. `4 > 3 && 10 < 12`
2. `4 > 3 && 10 > 12`
3. `4 > 3 || 10 < 12`
4. `4 > 3 || 10 > 12`
5. `!(4 > 3)`
6. `!(4 < 3)`
7. `!(false)`
8. `!(4 > 3 && 10 < 12)`
9. `!(4 > 3 && 10 > 12)`
10. `!(4 === '4')`

II- Les Bases de Javascript

Solution

```
> 4 > 3 && 10 < 12
```

```
↳ true
```

```
> 4 > 3 && 10 > 12
```

```
↳ false
```

```
> 4 > 3 || 10 < 12
```

```
↳ true
```

```
> 4 > 3 || 10 > 12
```

```
↳ true
```

```
> !(4 > 3)
```

```
↳ false
```

```
> !(4 < 3)
```

```
↳ true
```

```
> !(false)
```

```
↳ true
```

```
> !(4 > 3 && 10 < 12)
```

```
↳ false
```

```
> !(4 > 3 && 10 > 12)
```

```
↳ true
```

```
> !(4 === '4')
```

```
↳ true
```